



KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 2 Quiz

Fonksiyonlar ve Bellek Modeli

Doç. Dr. Tolga Berber · <https://avesis.ktu.edu.tr/tberber>

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

Soru 1 (10 puan)

4 • Değerlendirme Hedef 1

Aşağıdaki iki fonksiyon tanımı aynı çeviri biriminde birlikte bulunabilir mi?

```
1 int hesapla(int a, int b);
2 double hesapla(int a, int b);
```

- (a) Evet, derlenir; dönüş tipleri farklı olduğundan derleyici hangisinin çağrılacağını, sonucun atandığı değişkenin tipinden çıkarır.
- (b) Hayır, derlenmez; bir fonksiyonun imzasını ad ve parametre listesi belirler, dönüş tipi imzaya dahil değildir — bu iki tanım aynı imzaya sahip olduğundan derleyici “yeniden tanımlama” hatası verir.
- (c) Evet, derlenir; C++’ta aşırı yükleme yalnızca dönüş tipine bakılarak da yapılabilir.
- (d) Hayır, derlenmez; çünkü C++’ta aynı isimli iki fonksiyon hiçbir koşulda tanımlanamaz.

Soru 2 (10 puan)

4 • Değerlendirme Hedef 1

Aşağıdaki fonksiyon bildirimlerinden hangisi DERLENİR?

- (a) void ayarla(int a = 0, int b, int c);
- (b) void ayarla(int a, int b = 5, int c = 10);
- (c) void ayarla(int a = 0, int b = 1, int c);
- (d) void ayarla(int a, int b, int c = 1, int d);

Soru 3 (10 puan)

4 • Değerlendirme Hedef 2

`geometri::Nokta` nesnesini alıp yalnızca `x` ve `y` alanlarının toplamını döndüren, nesneyi hiçbir şekilde değiştirmeyecek bir fonksiyon yazacaksınız; gereksiz kopyalamadan da kaçınmak istiyorsunuz. Parametre hangi biçimde geçirilmelidir?

- (a) `Nokta n` (değere göre)
- (b) `Nokta& n` (referansla)
- (c) `const Nokta& n` (const referansla)
- (d) `Nokta* n` (işaretçiyle)

Soru 4 (10 puan)**4 • Değerlendirme** Hedef 3

`new Nokta(...)` ifadesinin, C'nin `(Nokta*)malloc(sizeof(Nokta))` ifadesine kıyasla sağladığı temel avantaj nedir?

- (a) `new` her zaman `malloc`'tan daha az bellek kullanır.
- (b) `new` tip güvenlidir (doğru tipte işaretçi döner, cast gerektirmez) ve nesnenin ilkendirilmesini otomatik çalıştırır; `malloc` yalnızca ham, tipsiz bellek ayırır ve hiçbir ilkendirme yapmaz.
- (c) `new` ile ayrılan bellek, işaretçi kapsam dışına çıktığında otomatik olarak geri alınır; bu yüzden `delete` çağırma hiçbir zaman gerek yoktur.
- (d) `new` ve `malloc` arasında yalnızca sözdizimsel bir fark vardır; ikisi de birebir aynı şekilde davranır.

Soru 5 (15 puan)**4 • Değerlendirme** Hedef 2

Aşağıdaki kodun ekrana ne bastığını yazınız.

```

1 struct Nokta { double x, y; };
2
3 void oteleDeger(Nokta n) { n.x = n.x + 10.0; }
4 void oteleReferans(Nokta& n) { n.x = n.x + 10.0; }
5
6 int main() {
7     Nokta a{1.0, 1.0};
8     oteleDeger(a);
9     std::cout << a.x << " ";
10    oteleReferans(a);
11    std::cout << a.x;
12 }
```

Soru 6 (15 puan)**4 • Değerlendirme** Hedef 3

Aşağıdaki fonksiyonu inceleyiniz. Bu fonksiyon çalıştırıldığında bir bellek sızıntısı oluşur mu? Oluşursa nedenini satır satır açıklayınız.

```

1 void isle() {
2     Nokta* p = new Nokta{1.0, 1.0};
3     p = new Nokta{2.0, 2.0};
4     delete p;
5 }
```

Soru 7 (15 puan)**4 • Değerlendirme** Hedef 3

Aşağıdaki program çalıştırıldığında ne olur? Açıklayınız ve fonksiyonu güvenli hale getirmek için nasıl bir düzeltme yapılması gerektiğini belirtiniz.

```

1 void yazdirDegeri(Nokta* p) {
2     std::cout << p->x;
3 }
```

Soru 7 (devamı)

```
4  
5 int main() {  
6     Nokta* p = nullptr;  
7     yazdirDegeri(p);  
8 }
```

Soru 8 (15 puan)**4 • Değerlendirme** Hedef 2

Bir fonksiyon parametresini değere göre, referansla ve işaretçiyle geçirmek arasındaki farkları açıklayınız. Büyük bir nesneyi yalnızca okumak (değiştirmemek) istediğinizde neden genellikle `const T&` tercih edilir?

Soru 9 (15 puan)**4 • Değerlendirme** Hedef 4

Bellek sızıntısı (*memory leak*) nedir? RAII ilkesi (bu haftaki ilk bakış düzeyinde) bunu nasıl önler? Yanıtınızda `araclar::IntTampon` örneğine değininiz.
